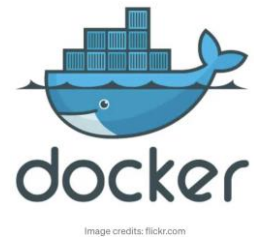


Docker Registry

Einstieg



Bemerkungen

In diesem Dokument werden Sie versuchen, eines der verbreitetsten Tools zum Containerisieren und Bereitstellen von Anwendungen über das Internet zu verstehen. Es macht die Bereitstellung von Anwendungen sehr einfach.

Bemerkungen

1	EINSTIEG IN DOCKER REGISTRY	2
1.1	EINLEITUNG	2
1.2	KOMPETENZEN	2
1.3	WAS IST DOCKER-HUB?	2
1.4	DOCKER-ENGINE/DEAMON	2
1.5	FOCUS DER FOLGENDEN AUFGABEN	3
1.6	AUFGABE 1 - EIN IMAGE VORBEREITEN	3
1.7	ÜBERSICHT ZU IMAGE ERSTELLEN UND ÜBERTRAGEN	4
1.8	AUFGABE 2 - IMAGE PUBLIZIEREN	4
1.9	AUFGABE 3 – PARTNERARBEIT: IMAGE LADEN, CONTAINER ERSTELLEN	6
1.10	AUFGABE 4 – UPDATE MIT VERSIONSÄNDERUNG	6
1.11	RÜCKBLICK	6

1 Einstieg in Docker Registry

1.1 Einleitung

In diesem Dokument lernen Sie eines der verbreitetsten Tools zum Containerisieren und Bereitstellen von Anwendungen über das Internet kennen. Docker-Hub macht die Bereitstellung von Anwendungen bzw. Images sehr einfach.

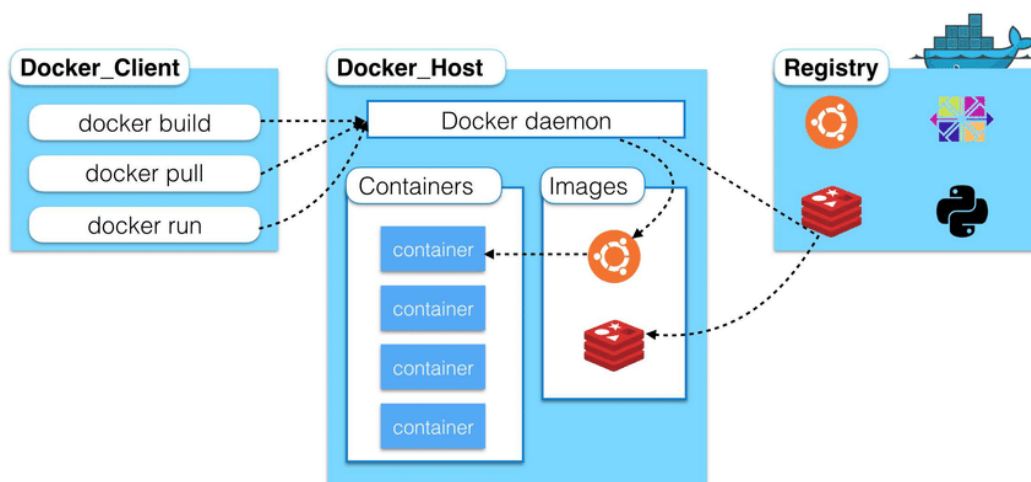
1.2 Kompetenzen

- Ich kann eine Anwendung in einem Image für das Laden auf Docker-Hub-Registry bereitstellen.
- Ich kann eine Anwendung bzw. ein Image benennen und die Version festlegen.
- Ich kann ein Image, versionsspezifisch, auf die Docker-Hub-Registry hochladen.
- Ich kann ein Image, versionsspezifisch, von der Docker-Hub-Registry herunterladen.
- Ich kann nicht mehr benötigte Images vom lokalen Rechner entfernen.
- Ich kenne die CLI-Docker-Befehle und kann diese anwenden.

1.3 Was ist Docker-Hub?

Docker-Hub ist eine gehostete Registry-Lösung, die von der Firma Docker Inc. bereitgestellt wird, und dadurch erhalten die Docker-Benutzer die Möglichkeit, auf öffentliche Open-Source-Image-Repositories zuzugreifen. Die Benutzer können verschiedene Docker-Image herunterladen, welche sie benötigen, diese anpassen und angepasste Images zurückschieben.

Der Docker-Hub gilt als das grösste Repository und verfügt zurzeit über mehr als 250.000 Repositories und Milliarden von Downloads.

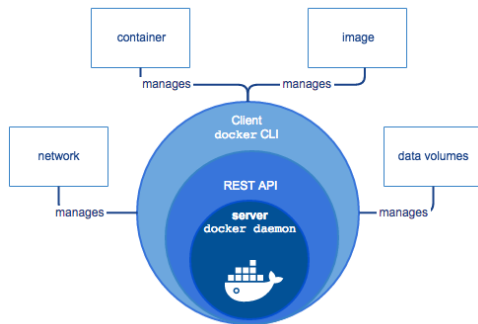


Docker Landscape (Credits: .jpaul.me)

1.4 Docker-Engine/Deamon

Die Software, die die Container hostet, heisst Docker Engine. Docker Engine ist eine Client-Server-basierte Anwendung, die aus 3 Hauptkomponenten besteht.

- Docker daemon: Er ist verantwortlich für das Erstellen und Verwalten von Docker-Images, Containern, Netzwerken und Volumes auf dem Docker. Er wird als Dämon-Prozess bezeichnet.
- REST-API: Sie gibt an, wie die Anwendungen mit dem Server interagieren können, und weist ihn an, was zu tun ist.
- Docker-Client: Der Client ist eine Docker-Befehlszeilenschnittstelle (CLI), die es uns ermöglicht, mit Docker mithilfe der Docker-Befehle zu interagieren.



Components of Docker (Credits: Docker.com)

1.5 Focus der folgenden Aufgaben

In den folgenden Aufgaben geht es darum, eigene Docker-Images zu erstellen und vom Docker-Host (dem eigenen Computer in eine Registry zu übertragen.

1.6 Aufgabe 1 - Ein Image vorbereiten

Das folgende Dockerfile basiert auf dem Nginx-Image und kopiert die HTML- und CSS-Dateien in das Nginx-Verzeichnis des Docker-Images. Ausserdem wird Port 80 des Docker-Containers veröffentlicht, um auf die Anwendung zugreifen zu können. Der Standardbefehl, der bei der Ausführung des Docker-Containers ausgeführt wird, startet den Nginx-Server.

→ Erstellen Sie einen Ordner im Arbeitsbereich, das folgende Dockerfile sowie die HTML- und die CSS-Datei.

Dockerfile

```
FROM nginx:alpine
MAINTAINER Your Name and E-Mail Address
COPY index.html /usr/share/nginx/html
COPY style.css /usr/share/nginx/html
EXPOSE 80
ENTRYPOINT ["nginx", "-g", "daemon off;"]
```

Index.html

```
<!DOCTYPE html>
<html>
  <head>
    <title>Meine HTML-Seite</title>
    <link rel="stylesheet" type="text/css" href="style.css">
  </head>
  <body>
    <h1>Willkommen auf meiner HTML-Seite</h1>
    <p>Dies ist ein Beispieltext.</p>
    <small>Version 1.0</small>
  </body>
</html>
```

style.css

```
body { background-color: lightblue; font-family: Arial, sans-serif; }
h1 { text-align: center; color: navy; }
p { font-size: 20px; text-align: justify; }
```

1.7 Übersicht zu Image erstellen und übertragen

Im Folgenden sehen Sie den groben Ablauf für das Erstellen und Pushen eines Images auf Docker-Hub. Lesen Sie im nächsten Kapitel die Details zu den einzelnen Schritten.

1. Erstellen Sie auf Docker-Hub einen Account (mit der E-Mail-Adresse der Schule).
2. Erzeugen Sie mit der Anweisung „Create Repository“ ein Docker-Hub-Repository.
3. Image bauen und markieren «taggen».
4. Pushen Sie das Image in das Docker-Hub-Repository.
5. Lokaler Image-Cache aufräumen.
6. Image vom Docker-Hub Repository laden.
7. Container vom neuen Image laufen lassen.



1.8 Aufgabe 2 - Image publizieren

Schritt 1: Account auf Docker-Hub erstellen

Damit Sie auf Docker-Hub eigene Images hochladen können müssen Sie registriert sein. Registrieren Sie sich auf Docker-Hub mit der E-Mail-Adresse der Schule. Die Registrierung und die Publikation von Images sind kostenlos, während Docker für «private» Images, die einen Zugriffsschutz besitzen, eine monatliche Gebühr verlangt <https://hub.docker.com>.

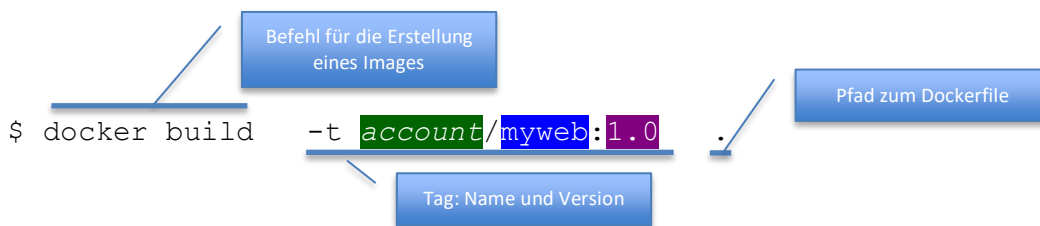
Schritt 2: Repository für das Image erstellen

Nun sind Sie registriert und können eigene Images erstellen und auf Docker-Hub hochladen. Bei der Erzeugung des Repository müssen Sie den **Account-Namen** für die Identifikation des Images angeben und sich merken.

→ Erzeugen das Repository z.B. mit dem Namen **myweb**

Schritt 3: Image bauen und taggen

Der folgende Befehl dient der Erstellung des Images sowie der Benennung und Spezifizierung der Version.



Achten Sie darauf, dass Sie dem Image den Namen Ihres Accounts voranstellen und mit einem / trennen.

Überprüfen Sie nun die korrekte Erstellung des Images mit folgendem Befehl

```
$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
account/myweb	1.0	03c0f96371c7	45 seconds ago	40.3MB

Schritt 4: Image auf Docker-Hub-Repository pushen

Um ein Image auf das Remote Repository zu pushen, müssen Sie sich zuerst über die Docker-CLI bei Docker-Hub anmelden. Dies geschieht über den unten folgenden Befehl. Geben Sie bei der Aufforderung den Benutzernamen und das Passwort ein, um sich anzumelden.

```
$ docker login
```

Danach können Sie das Image auf das Remote Repository pushen

```
$ docker push account/myweb:1.0
```

Nun sollten Sie in Docker-Hub in der Liste Ihrer Repository unter **myweb** das gerade gepushte Image finden.

The screenshot shows the Docker Hub interface for a repository named 'myweb' under the username 'bbwrl'. The repository is described as a 'teaching example' and was last pushed 21 minutes ago. On the right, the 'Docker commands' section shows the command 'docker push bbwrl/myweb:tagname'. Below this, the 'Tags' section indicates that the repository contains 1 tag(s). A table lists the tag '1.0' with its OS, Type (Image), Pulled time (4 minutes ago), and Pushed time (22 minutes ago). There are links for 'See all' and 'Go to Advanced Image Management'. On the right side, the 'Automated Builds' section explains how to connect to GitHub or Bitbucket for automatic builds, with an 'Upgrade' button and a 'Learn more' link.

Tag	OS	Type	Pulled	Pushed
1.0		Image	4 minutes ago	22 minutes ago



Credits: VectorStock.com/27158023

Schritt 5: Lokaler Image-Cache aufräumen

Bevor Sie nun weiterfahren, räumen Sie den lokalen Cache mit den gespeicherten Images auf. Verschaffen Sie sich zuerst einen Überblick mit den entsprechenden Befehlen.

Löschen Sie die/das geladene Image umsichtig: Löschen Sie die Images aus Modul 347.

Bei den anderen Images überlegen Sie, ob diese in anderen Modulen noch verwendet werden und entscheiden dann über das Löschen.

```
$ docker images
```

```
$ docker image rm {id}
```

Falls ein Image nicht gelöscht werden kann, liegt dies vermutlich an Containern die noch aktiv sind und das Image benötigen. Überlegen Sie, ob Sie diesen Container noch benötigen. Wenn nicht, dann stoppen Sie betroffene Container und löschen Sie diese. Versuchen Sie danach erneut das Löschen dieser nicht mehr benötigter Images.

1.9 Aufgabe 3 – Partnerarbeit: Image laden, Container erstellen

Schritt 6: Image von Docker-Hub-Repository laden

So, nun haben Sie Ihr erstes Image auf Docker-Hub hochgeladen. Da dieses Image «public» publiziert wurde, steht dieses nun allen zur Verfügung.



→ Teilen Sie den Namen und die Version Ihrer Nachbarin/Ihrem Nachbarn mit.

→ Versuchen Sie das Image des jeweils anderen herunterzuladen.

```
$ docker pull {name}:{version}
```

Schritt 7: Container von Image laufen lassen

Untersuchen Sie mit den Ihnen bekannten Docker-Befehlen das Image und starten Sie das Image in einem Container.

Öffnen Sie die Ansicht der Applikation aus dem Container im Browser.

1.10 Aufgabe 4 – Update mit Versionsänderung

In dieser Aufgabe geht es darum ein Update des eigenen Images mit einer Versionsänderung durchzuführen. Dabei wird in der Docker Registry eine neue Version des Images angelegt und die alte behalten, sodass diese theoretisch weiterhin geladen werden könnte.

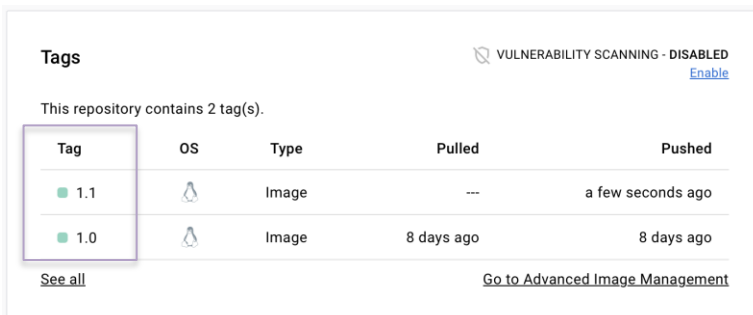
→ Ändern Sie in der HTML-Datei den Text im `<small>`-Tag, um die Änderung zu visualisieren.

```
<small>Version 1.1</small>
```

→ Bauen Sie das neue Image mit dem Versions-Tag 1.1

→ Pushen Sie das neue Image in die Registry auf Docker-Hub

→ Kontrollieren Sie in der Registry, ob beide Versionen Ihres Images vorhanden sind



The screenshot shows the 'Tags' page in the Docker Registry. It indicates that the repository contains 2 tags. A table lists the tags with columns for Tag, OS, Type, Pulled, and Pushed. Tag 1.1 is highlighted with a red box. Tag 1.0 is also listed. There is a 'VULNERABILITY SCANNING - DISABLED' warning at the top right and a 'See all' link at the bottom left.

Tag	OS	Type	Pulled	Pushed
1.1	linux/amd64	Image	---	a few seconds ago
1.0	linux/amd64	Image	8 days ago	8 days ago

1.11 Rückblick

Sie haben die Docker Registry kennengelernt, eine Anwendung mit Docker erstellt und ein eigenes Image an Docker-Hub gepusht und dieses Image wieder auf den lokalen Rechner geholt.

Ausserdem haben Sie gelernt, wie Sie ein bestimmtes Image von unserem lokalen Rechner entfernen und das Image später aus Docker-Hub ziehen können.

Sie haben gelernt, wie Docker-Images mit Versionen getagt werden, um diese bei Verwendung genau zu identifizieren.